

วงจรบวกค่านวนผลรวมสะสมจาก n ถึง 1

(Sum n down to 1)

ด้วยโปรแกรม Logisim-Evolution

ระบบการออกแบบหน่วยประมวลผลขนาดเล็กแบบแยกส่วนควบคุม (Control Unit) และส่วนการไหลของข้อมูล (Datapath) ที่ทำงานร่วมกับ Register File และ ALU 8 bit

จัดทำโดย

- | | | |
|---|---------------------------|------------|
| 1 | นายดรัณภพ พิทักษ์กิจไพศาล | 6710301007 |
| 2 | นายอภิศักดิ์ คงภักดี | 6710301009 |
| 3 | นายธนัท จงธีรธนโชติ | 6710301032 |

เสนอ อาจารย์คทาเทพ สวัสดิ์พิศาล

รายวิชา Embedded System Design ภาคการศึกษาที่ 1 ปีการศึกษา 2569

สารบัญ (Table of Contents)

1. บทนำ (Introduction)	3
2. วัตถุประสงค์ (Objectives)	3
3. เครื่องมือที่ใช้ (Tools Used)	3
4. แนวคิดการออกแบบวงจร (Circuit Design Concept)	4
4.1 โครงสร้างการส่งผ่านข้อมูล (Datapath Architecture)	4
4.2 รายละเอียดโครงสร้างการควบคุม FSM (Control Unit)	5
5. ขั้นตอนการสร้างวงจร (Implementation)	6
5.1 สร้างโมดูลหน่วยประมวลผลจำนวน (ALU 8 bit)	6
5.2 สร้างโมดูลบันทึก Register File (Register File)	6
5.3 ออกแบบสร้างวงจรควบคุมสถานะ (Control Unit)	7
5.4 ประกอบแผงวงจรหลักและการทดสอบ	7
6. ผลการทดสอบ (Testing Results)	8
7. ปัญหาที่พบและวิธีแก้ไข (Problems & Solutions)	9
8. สรุปผล (Conclusion)	10
9. ภาคผนวก (Appendix)	10

1 บทนำ (Introduction)

การเขียนโปรแกรมเพื่อคำนวณผลรวมสะสมจากค่า n ลงมาถึง 1 เช่น $(5 + 4 + 3 + 2 + 1 = 15)$ เป็น Function พื้นฐานที่มักจะใช้วิธีวนลูป (Loop) ในระดับ Software แต่สำหรับระบบ Computer และระบบสมองกลฝังตัว (Embedded System) เราสามารถสร้างกลไกการคำนวณนี้ขึ้นมาได้ด้วยวงจร Hardware จำลอง โดยเชื่อมต่อส่วนประมวลผลและหน่วยความจำเข้าด้วยกัน ซึ่งจะช่วยให้เข้าใจสถาปัตยกรรม Computer และกลไกการทำงานในระดับ Hardware ได้อย่างลึกซึ้ง

รายงานฉบับนี้เสนอผลการออกแบบและสร้างวงจรคำนวณผลรวมสะสม n ถึง 1 (Sum n down to 1) ขนาด 8 bit บนโปรแกรม Logisim-Evolution โดยจัดโครงสร้างแบบแยกส่วนควบคุม (Control Unit) และส่วนข้อมูล (Datapath) ที่ทำงานสอดประสานกันตามสัญญาณนาฬิกา (Synchronous Clock) ผ่านทาง Register File และหน่วยคำนวณและตรรกะ (ALU)

การทดลองนี้จะช่วยให้เข้าใจกลไกการทำงานของหน่วยประมวลผลจริง ทั้งการจัดการ Data Bus การส่งสัญญาณควบคุมจาก FSM (Finite State Machine) การอ่านเขียน Register ตลอดจนการส่งผลลัพธ์ออกจาก Output Pin เมื่อคำนวณเสร็จสิ้น

2 วัตถุประสงค์ (Objectives)

1. เพื่อออกแบบและสร้างวงจรคำนวณผลรวมสะสมจาก n ถึง 1 (Sum n down to 1) ด้วยโปรแกรม Logisim-Evolution
2. เพื่อศึกษาโครงสร้างและจังหวะการทำงานสอดประสานกันระหว่างส่วนควบคุม (Control Unit) และส่วนข้อมูล (Datapath) ผ่าน Register File และหน่วยคำนวณและตรรกะขนาด 8 bit
3. เพื่อวิเคราะห์ความถูกต้องและพฤติกรรมเวลาของหน่วยประมวลผลผ่านกราฟสัญญาณเวลา (Chronogram)

3 เครื่องมือที่ใช้ (Tools Used)

3.1 โปรแกรมจำลองการทำงาน

Logisim-Evolution v4.1.0 — โปรแกรมสำหรับออกแบบและทดสอบวงจร Logic ซึ่งในโปรเจกต์นี้ใช้ในการสร้างสถาปัตยกรรมหน่วยประมวลผลย่อย ๆ และวิเคราะห์กราฟสัญญาณเวลา

3.2 ส่วนประกอบและโมดูลย่อยหลัก (Subcircuits)

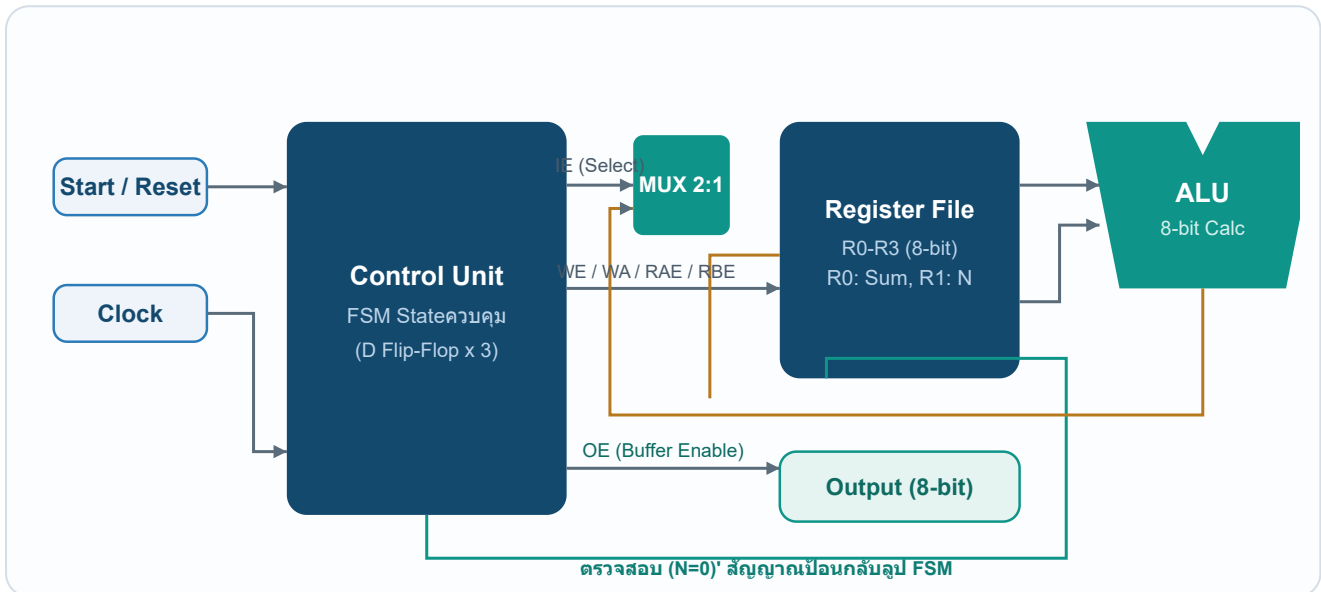
ตารางที่ 1 โมดูลย่อยหลักที่ใช้ภายในวงจรและบทบาทหน้าที่

อุปกรณ์ / โมดูล	หน้าที่ในวงจร
Control Unit (FSM)	ควบคุมการทำงานในแต่ละState สร้างสัญญาณควบคุมสั่งงาน Datapath และส่งสัญญาณแจ้งเตือนเมื่อคำนวณเสร็จสิ้น
Register File (Four_Bit_RF)	หน่วยความจำขนาด 8 bit จำนวน 4 Register (R0-R3) รองรับการอ่านและเขียนข้อมูลพร้อมกัน
ALU (Arithmetic Logic Unit)	หน่วยประมวลผลคำนวณ รองรับการบวกค่าตัวเลขสะสม (ADD) และลดค่าตัวนับลงทีละหนึ่ง (DEC)
Multiplexer 8 bit	สลับสัญญาณขาเข้าของ Register File ระหว่างค่าเริ่มต้นของข้อมูลInputกับค่าคำนวณใหม่จาก ALU
Controlled Buffer	เป็น Tri-State Buffer ที่ทำหน้าที่เปิดและส่งข้อมูลOutputสะสมออกสู่ภายนอกเมื่อ Done สิ้นสุดลง
Clock & Start / Reset Pins	ปุ่มInputรับสัญญาณควบคุมจังหวะและคำสั่งการเริ่มต้นและการล้างระบบวงจรจากภายนอก

4 แนวคิดการออกแบบวงจร (Circuit Design Concept)

4.1 โครงสร้างการส่งผ่านข้อมูล (Datapath Architecture)

โครงสร้างวงจรนี้ออกแบบเป็นระบบคำนวณขนาด 8 bit เพื่อรองรับการบวกสะสมค่าที่ใหญ่ขึ้น โดยฝั่ง Datapath จะมี Register File (Four_Bit_RF) ทำหน้าที่เก็บข้อมูลหลัก ซึ่งใช้ **R0** เก็บผลรวมสะสม (Sum) และใช้ **R1** เก็บตัวนับ Input (N) จากนั้นจะมี **ALU** เป็นตัวประมวลผลหลักในการบวกสะสมและลดค่าตัวนับทีละ 1



รูปที่ 1 Block Diagramของระบบคำนวณ Sum n down to 1 — แสดงการเชื่อมต่อระหว่าง Control Unit และ Datapath ขนาด 8 bit

4.2 รายละเอียดโครงสร้างการควบคุม FSM (Control Unit)

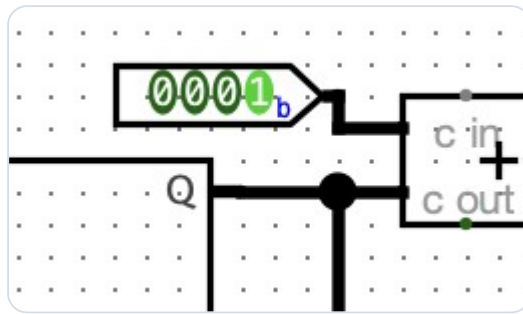
การทำงานของส่วนควบคุม (Control Unit) จะใช้ D Flip-Flop จำนวน 3 ตัวเพื่อสร้างสถานะ (State) ของ Finite State Machine (FSM) โดยการเข้ารหัสสถานะจริงในวงจร (FSM State Encoding) ได้รับการออกแบบให้สอดคล้องกับพฤติกรรมการควบคุมของ Datapath ดังนี้:

- **State 000 (Idle/Reset)** — สถานะเริ่มต้นเพื่อคอยสัญญาณ **Start** หากมีการกดปุ่ม Start วงจรจะเปลี่ยนStateเพื่อเริ่มโหลดค่า
- **State 100 (Load Data)** — สั่งการเขียนค่าเริ่มต้น **Data Input** ลงRegister R1 (เก็บค่า N) ผ่านทางสัญญาณควบคุม IE=1, WE=1, WA=01 โดยมีข้อจำกัดที่วงจรไม่ได้จัดส่งสัญญาณล้างค่าเริ่มต้นให้แก่ R0 (Sum) ในขั้นตอนนี้ (ต้องสั่ง Reset Simulation ของ Logisim-Evolution เพื่อ clear R0 เป็น 0 ก่อนเริ่มต้นการประมวลผลเสมอ)
- **State 010 (Check & Add)** — ตรวจสอบค่า N ผ่านขา (**N=0**)' หาก N ไม่เท่ากับ 0 จะสั่งบวกสะสม $R0 = R0 + R1$ โดยใช้คำสั่งอ่าน RAE=1, RAA=00, RBE=1, RBA=01 ส่งไปที่ ALU และเลือกคำสั่ง ALU ADD (รหัส `100` ใน Multiplexer จริง) จากนั้นเขียนกลับลง R0
- **State 110 (Decrement N)** — สั่งลบค่าตัวนับ $R1 = R1 - 1$ โดยสั่งอ่าน RAE=1, RAA=01 เลือกคำสั่ง ALU DEC (รหัส `111` ใน Multiplexer จริง) และสั่งบันทึกข้อมูลเขียนกลับลง R1 (WE=1, WA=01) จากนั้นวนกลับไปยังStateตรวจสอบ
- **State 001 (Done)** — เมื่อ N มีค่าเป็น 0 สัญญาณแจ้งเตือน (**N=0**)' จะเปลี่ยนเป็น 0 วงจรจะเปลี่ยนมายังStateเสร็จสิ้น สั่งงานขา Done=1 เพื่อแจ้งเสร็จงาน และ OE=1 เพื่อปลดปล่อยผลลัพธ์ผ่าน Buffer ออกภายนอก
- **State 101 (Auto-Reset Transition)** — Stateเปลี่ยนผ่านชั่วคราวเพื่อResetวงจรกลับสู่สถานะเริ่มต้นเมื่อสิ้นสุดการคำนวณและผู้ใช้กดปุ่ม Start ซ้ำอีกรอบ

5 ขั้นตอนการสร้างวงจร (Implementation)

5.1 สร้างโมดูลหน่วยประมวลผลคำนวณ (ALU 8 bit)

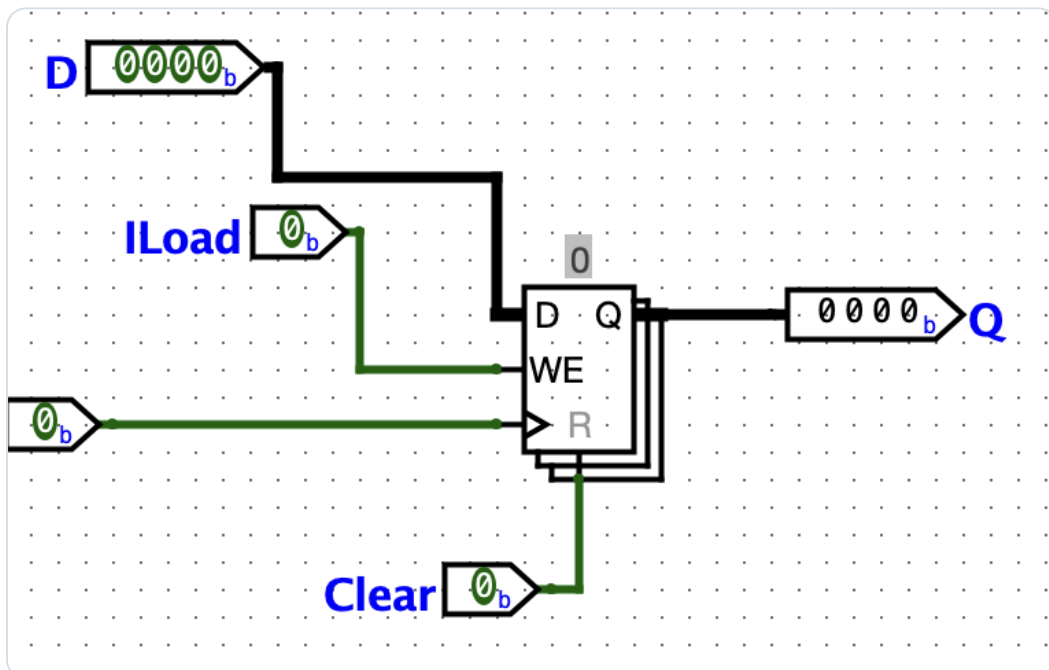
สร้างส่วนคำนวณหลักด้วยอุปกรณ์ Adder และ Subtractor ขนาด 8 bit นำผลลัพธ์ผ่าน Multiplexer 8 ช่องเพื่อคัดเลือกรหัสคำสั่งผ่านขาควบคุม 3 เส้น (ALU2, ALU1, ALU0) โดยเน้นFunctionคำนวณบวก สะสมและการลบทีละหนึ่ง



รูปที่ 2 วงจรจำลองภายในโมดูล ALU ขนาด 8 bit — บล็อกคำนวณหลักและการเลือกFunctionด้วย Multiplexer

5.2 สร้างโมดูลบันทึกRegister File (Register File)

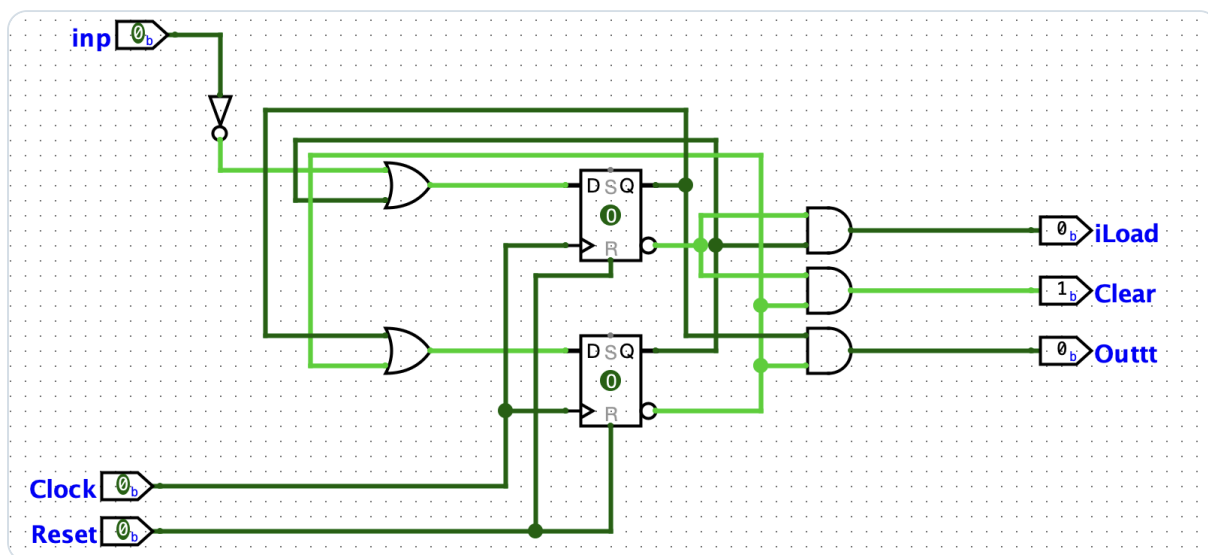
ต่อวงจรRegister 8 bit จำนวน 4 ตัวเพื่อใช้เก็บสถานะข้อมูล ใช้ตัวถอดรหัส (Decoder) ในการถอดรหัสเลือกเขียน (WA) และอ่านออกPortพร้อมกัน 2 ช่องสัญญาณ (Port A และPort B)



รูปที่ 3 โครงสร้างการเชื่อมต่อภายใน Register File ขนาด 8 bit — แสดงการจัดPortอ่านเขียนRegister R0-R3

5.3 ออกแบบสร้างวงจรควบคุมสถานะ (Control Unit)

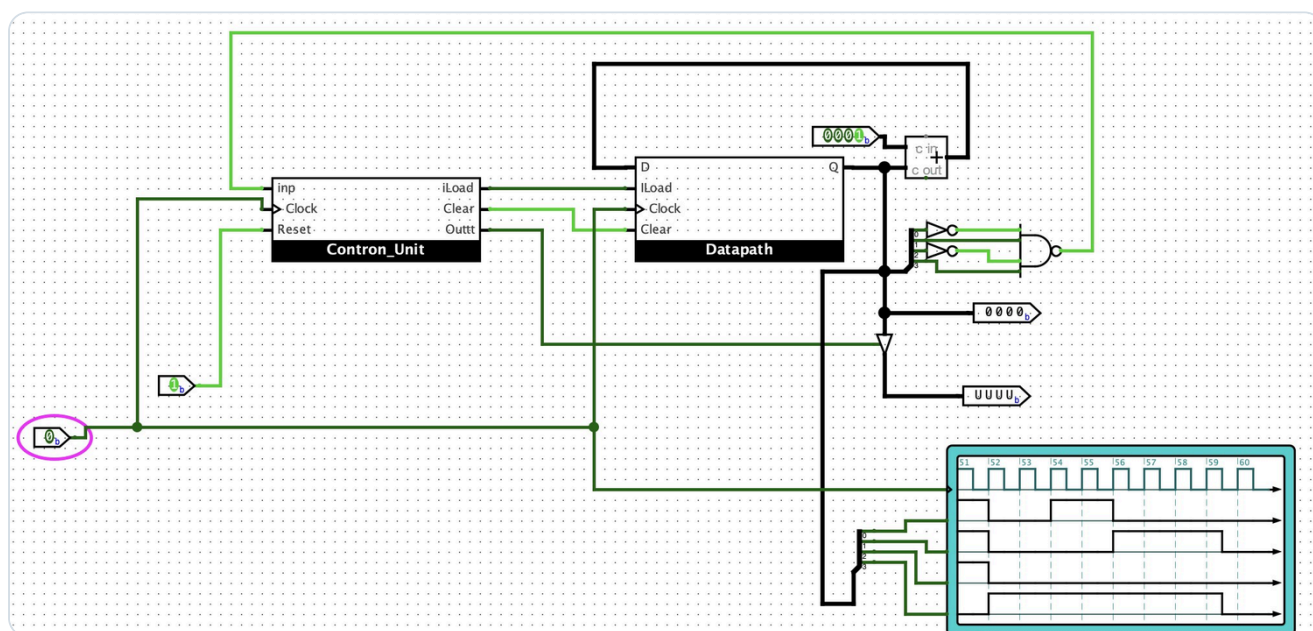
สร้างวงจร FSM หลักด้วย D Flip-Flop จำนวน 3 ตัวในการระบุสถานะ ต่อ Logic Gate เพื่อแปลงค่าป้อนกลับทางสถานะและสัญญาณขา Output (Control Signals) ให้ทำงานพร้อมเพรียงกันในแต่ละ Cycle



รูปที่ 4 แผง Logic ภายในส่วน Control Unit — FSM ควบคุมสัญญาณนาฬิกาและขาคำสั่งควบคุม Datapath

5.4 ประกอบแผงวงจรหลักและการทดสอบ

นำวงจรควบคุม ส่วนข้อมูล สัญญาณ Input และตัวดำเนินการสถานะ Output มารวมต่อเข้าด้วยกันในวงจรหลัก จากนั้นเชื่อมสัญญาณต่าง ๆ เข้าสู่เครื่องวัดสัญญาณ Chronogram เพื่อวิเคราะห์พฤติกรรมการเปลี่ยนจังหวะของ Logic ในแต่ละสถานะได้อย่างแม่นยำ



รูปที่ 5 วงจรคำนวณผลรวมสะสม Sum n down to 1 ที่เชื่อมต่อเสร็จสมบูรณ์บน Logisim-Evolution

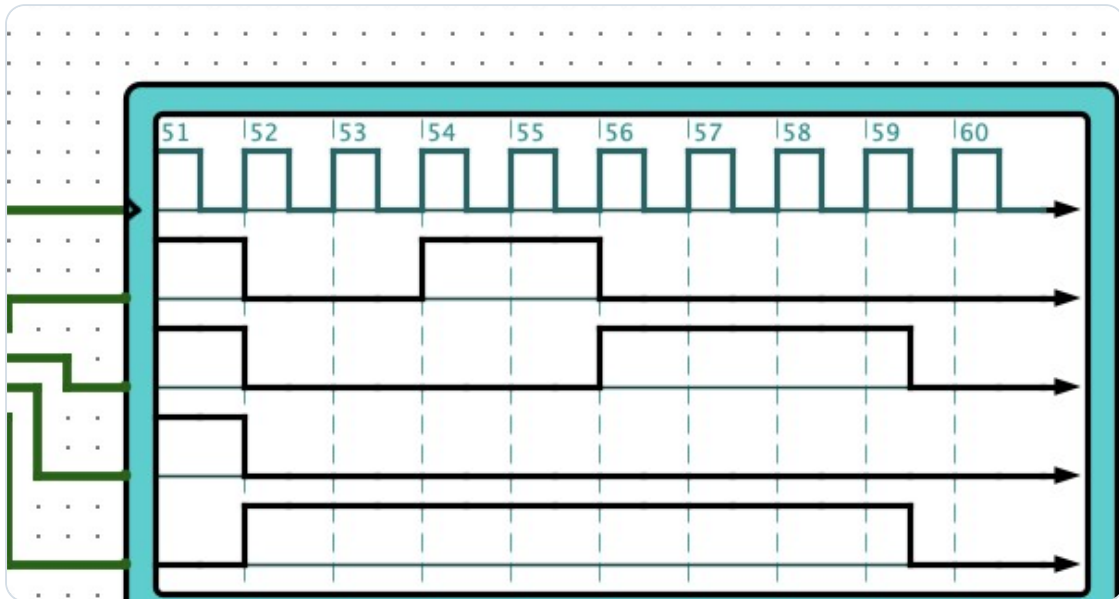
6 ผลการทดสอบ (Testing Results)

การทดสอบจำลองประสิทธิภาพระบบโดยระบุInputเริ่มต้นที่ $n = 5$ (คาดหวังผลลัพธ์ผลรวมเท่ากับ 15 หรือ 0000 1111 ในเลขฐานสอง) จากนั้นทำการ Auto-Tick Clock ทีละCycle เพื่อประเมินค่าข้อมูลภายในRegister R0 (Sum) และ R1 (N) ในแต่ละจังหวะเวลาทำงาน:

ตารางที่ 2 การวิเคราะห์Stateการทำงานและการเปลี่ยนค่าในแต่ละCycle ($n = 5$)

จังหวะ Clock	State FSM (Q2 Q1 Q0)	ค่าใน R0 (Sum)	ค่าใน R1 (Counter N)	พฤติกรรมของวงจรและคำสั่ง
0	000 (Idle)	0000 0000	0000 0000	คอยคำสั่ง เริ่มการประมวลผล
1	100 (Load)	0000 0000	0000 0101	โหลดค่าInput $n=5$ ลง R1 (R0=0 clearLogicจำลอง)
2	010 (Add)	0000 0101	0000 0101	บวกสะสม: $R0 = R0 + R1$ ($0 + 5 = 5$)
3	110 (Dec)	0000 0101	0000 0100	ลดค่าตัวนับ: $R1 = R1 - 1$ ($5 - 1 = 4$)
4	010 (Add)	0000 1001	0000 0100	บวกสะสม: $R0 = R0 + R1$ ($5 + 4 = 9$)
5	110 (Dec)	0000 1001	0000 0011	ลดค่าตัวนับ: $R1 = R1 - 1$ ($4 - 1 = 3$)
6	010 (Add)	0000 1100	0000 0011	บวกสะสม: $R0 = R0 + R1$ ($9 + 3 = 12$)
7	110 (Dec)	0000 1100	0000 0010	ลดค่าตัวนับ: $R1 = R1 - 1$ ($3 - 1 = 2$)
8	010 (Add)	0000 1110	0000 0010	บวกสะสม: $R0 = R0 + R1$ ($12 + 2 = 14$)
9	110 (Dec)	0000 1110	0000 0001	ลดค่าตัวนับ: $R1 = R1 - 1$ ($2 - 1 = 1$)
10	010 (Add)	0000 1111	0000 0001	บวกสะสมรอบสุดท้าย: $R0 = R0 + R1$ ($14 + 1 = 15$)
11	110 (Dec)	0000 1111	0000 0000	ลดค่าตัวนับ: $R1 = R1 - 1$ ($1 - 1 = 0$)
12	001 (Done)	0000 1111	0000 0000	เสร็จงาน แสดงผลลัพธ์เป็น 15 (0000 1111) ดำรงรอ Reset

จากการประเมิน วงจรสามารถจำลองและแสดงผลลัพธ์ได้ถูกต้องตามทฤษฎี และจะสิ้นสุดค้างสถานะOutputพิน Done ไว้เพื่อรอจนกว่าระบบจะได้รับสัญญาณ Reset จากผู้ใช้เพื่อล้างRegisterและกลับไปทำงานที่จุดเริ่มต้นอีกครั้ง



รูปที่ 6 กราฟสัญญาณการวัดจังหวะ (Chronogram) ขณะทำการประมวลผลคำนวณบวกสะสม

7 ปัญหาที่พบและวิธีแก้ไข (Problems & Solutions)

ปัญหา 7.1: ผลคำนวณสะสมในRegister R0 บวกเกินค่าจริง

วิธีแก้ไข: เกิดจากLogicการเปลี่ยนผ่าน FSM ในส่วนตรวจสอบเงื่อนไข N ผิดพลาด ทำให้เมื่อ $N=0$ แล้ววงจรยังบวกสะสมต่อไปอีกหนึ่งรอบCycle ได้ทำการปรับแก้จุดเปลี่ยนผ่านในBoard Control Unit ให้เชื่อมเงื่อนไขตรวจสอบจากRegister R1 ตรงมายังการตรวจค่า 0 ทันที ทำให้สามารถตัดกระบวนการได้อย่างแม่นยำ

ปัญหา 7.2: ข้อมูลในRegisterสูญหายในจังหวะสลับเฟสการเขียน

วิธีแก้ไข: เนื่องจากจังหวะควบคุมสัญญาณการบันทึก (WE) ของRegisterขัดแย้งกับเฟสสัญญาณนาฬิกาหลัก จึงจัดสรรวงจรใหม่โดยเชื่อมสัญญาณ CLK หลักเข้าตรงที่อุปกรณ์Registerทุกชุดโดยตรง และใช้สัญญาณควบคุมเฉพาะผ่านLogic Gateของหน่วย FSM เข้าสั่งเฉพาะขา Write Enable (WE) เท่านั้น

8 สรุปผล (Conclusion)

การออกแบบและทดสอบวงจรคำนวณผลรวมสะสมจาก n ถึง 1 ด้วยโปรแกรม Logisim-Evolution ประสบความสำเร็จตามที่ตั้งเป้าหมายไว้ วงจรย่อยทุกส่วน ทั้งหน่วยประมวลผล (ALU) Register File (Register File) และวงจร Logic FSM ในส่วนควบคุม ทำงานร่วมกันแบบสอดประสานได้อย่างถูกต้อง สามารถประมวลผลการคำนวณและแสดงค่าสะสมขนาด 8 bit ได้สมบูรณ์และถูกต้องทุกจังหวะเวลา

ผู้เรียนได้ศึกษาและนำความรู้การออกแบบระบบแยกส่วนควบคุมและส่วนข้อมูล (Control Unit / Datapath) มาใช้อย่างเป็นรูปธรรม เข้าใจกระบวนการจัดส่ง Data Bus การระบุสถานะทางคำสั่ง FSM การอ่านและบันทึกค่าลง Register ซึ่งเป็นแนวคิดแกนหลักที่นิยมนำมาพัฒนาต่อขยายไปสู่การออกแบบ Processor ในระบบ Computer และชิปสมองกลฝังตัวระดับสูงขึ้นไป

9 ภาคผนวก (Appendix)

รายละเอียดสัญญาณเสริมในวงจรคำนวณผลรวมสะสม n ถึง 1

ตารางที่ 3 รหัสเลือกการทำงานของ ALU (ALU Select) 8 Function

รหัส (ALU2-0)	การทำงานหน้าที่ (Function)	คำอธิบาย Logic
000	Pass A	ส่งผ่านข้อมูล A โดยไม่คำนวณ
001	AND	$A \text{ AND } B$ (Logic AND ระดับ bit)
010	OR	$A \text{ OR } B$ (Logic OR ระดับ bit)
011	NOT	NOT A (กลับ bit A ทั้งหมด)
100	ADD	$A + B$ (บวกข้อมูล 8 bit)
101	SUB	$A - B$ (ลบข้อมูล 8 bit)
110	INC	$A + 1$ (เพิ่มข้อมูลขึ้นหนึ่ง)
111	DEC	$A - 1$ (ลดข้อมูลลงหนึ่ง)